

LYCANS AERODESIGN TEAM

Training Task 06

Vision-Based Obstacle Avoidance & Command Generation

Deadline: Sep 2, 2026 - 11:59 PM

Submission: GitHub Desktop / Pull Request (task-06/)

1 Task / Mission

Welcome to the visual perception task for the Lycans Autonomous Flight Subsystem. In real-world UAV applications, an onboard camera serves as the aircraft's primary eye. Before deploying complex Machine Learning models, an engineer must master classical Computer Vision (CV) techniques to process image arrays, filter environmental noise, extract geometric features, and map detections directly into flight control logic.

1.1 Objective

In this introductory assignment, you will build a simple OpenCV-based vision system using Python that captures live video from a webcam, detects colored geometric markers in real time, determines their spatial position in the camera frame, and generates simulated directional flight commands.

Your key focus is building a robust processing pipeline following this continuous control loop:

Camera Feed → Color Conversion (HSV) → Shape Detection → Spatial Decision → Flight Command

Note: You are **NOT** required to use deep learning, YOLO, Roboflow, datasets, or bounding box annotations. This assignment evaluates your understanding of fundamental classical image processing.

2 Operational Scenario

Imagine a fixed-wing or multirotor aircraft equipped with a forward-facing optical camera cruising through a low-altitude corridor. As the aircraft moves ahead, its onboard vision system constantly scans the airspace for navigation targets and collision hazards.

The vision script must visually isolate objects based on color and geometric properties, compute their frame coordinates, and print real-time directional commands to the terminal.

IMPORTANT DISCLAIMER:

The program should NOT control a real aircraft. It will only print and simulate flight commands in the terminal.

3 Required Detections & Logic

Your algorithm must detect two specific visual targets and react according to their position and operational meaning.

3.1 Target Definitions

Table 1: Target Detections and System Response Matrix

Detected Object	Operational Meaning	System Status	Default Command
Red Square	Obstacle detected	OBSTACLE	MOVE DOWN
Green Circle	Safe flight corridor	SAFE	CONTINUE
Unknown / Other	Visual noise / uncertain	UNCERTAIN	HOLD
None	Open airspace	CLEAR	NO OBJECT DETECTED

3.2 Position-Based Evasive Behavior

The camera view frame must be logically divided horizontally into three equal spatial regions: **LEFT**, **CENTER**, and **RIGHT**.

- **When a Red Square (Obstacle) is detected:**
 - Detected in **LEFT** region → Status: OBSTACLE LEFT → Command: MOVE RIGHT
 - Detected in **CENTER** region → Status: OBSTACLE AHEAD → Command: MOVE DOWN
 - Detected in **RIGHT** region → Status: OBSTACLE RIGHT → Command: MOVE LEFT
- **When a Green Circle (Safe Path) is detected:**
 - Any region → Status: PATH CLEAR → Command: CONTINUE

4 Student Requirements

Your script must be written in Python using OpenCV (`cv2`) and follow these technical steps:

1. **Live Capture:** Capture live video stream from your machine's webcam.
2. **Color Space Conversion:** Convert raw RGB/BGR camera frames to the **HSV** (Hue, Saturation, Value) color space.
3. **Color Thresholding:** Segment the frame to isolate Red and Green regions using `cv2.inRange()`.
4. **Contour Extraction:** Find object boundaries using `cv2.findContours()` on the binary masks.
5. **Geometric Classification:** Determine whether the extracted contour represents a square or circle using contour properties (e.g., polygon approximation via `cv2.approxPolyDP` or circularity ratios).
6. **Bounding Visuals:** Draw a bounding box around the detected object on the live output feed.
7. **Center Calculation:** Compute and display the object's center coordinates (X_{center}, Y_{center}) on the frame.
8. **Frame Partitioning:** Determine whether the target's center lies in the **LEFT**, **CENTER**, or **RIGHT** portion of the frame.
9. **Terminal Logging:** Print appropriate simulated flight commands directly to the terminal.
10. **Display Output:** Show the live processed camera feed in a GUI window.

5 Suggested Approach & Hints

If you have little or no previous Computer Vision experience, break your program down into manageable steps:

5.1 Color Thresholding (BGR vs. HSV)

Standard images use BGR in OpenCV. Because lighting conditions change rapidly, HSV separates color identity (**Hue**) from color intensity (**Saturation**) and brightness (**Value**), making thresholding far more stable.

```
# Convert BGR to HSV
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# Define HSV range for green mask
lower_green = np.array([35, 100, 100])
upper_green = np.array([85, 255, 255])
mask_green = cv2.inRange(hsv, lower_green, upper_green)
```

5.2 Shape Approximation

After extracting contours using `cv2.findContours()`, filter out minor noise using `cv2.contourArea(cnt) > 500`:

- Use `cv2.approxPolyDP()` to approximate the number of polygon vertices.
- If the shape has **4 vertices**, classify it as a Square/Rectangle.
- If it has **more vertices** (e.g., > 6) and high circularity ($\frac{4\pi \cdot Area}{Perimeter^2} \approx 1.0$), classify it as a Circle.

5.3 Center Zone Logic

Extract coordinates using image moments and divide frame width (W) by 3:

```
M = cv2.moments(cnt)
cx = int(M['m10'] / M['m00'])
cy = int(M['m01'] / M['m00'])

if cx < width / 3:
    position = "LEFT"
elif cx < 2 * (width / 3):
    position = "CENTER"
else:
    position = "RIGHT"
```

6 Expected Output

Your terminal output should stream clear, formatted flight logs as objects move across the feed.

6.1 Sample Output 1: Obstacle Center

```
[CV] Red square detected  
[CV] Center: (612, 340)  
[CV] Position: CENTER  
[CV] Status: OBSTACLE AHEAD  
[CV] Command: MOVE DOWN
```

6.2 Sample Output 2: Safe Path

```
[CV] Green circle detected  
[CV] Center: (645, 355)  
[CV] Position: RIGHT  
[CV] Status: PATH CLEAR  
[CV] Command: CONTINUE
```

7 Testing

To test your vision pipeline:

1. Display or print a solid **Red Square** and a solid **Green Circle** on a screen or paper card.
2. Move the markers across your webcam frame from left to center to right.
3. Verify that terminal updates respond correctly to the target's relative position and geometric shape.

8 Bonus Challenges

Optional challenges for students who finish early:

- **Bonus 1 — Distance Approximation:** Use the bounding-box size (area or height/width) as a rough indication of whether the object is NEAR or FAR. Include a comment explaining why this is only a rough visual heuristic and NOT true 3D spatial range estimation.
- **Bonus 2 — Avoid Terminal Spam:** Instead of printing 30 times per second every frame, store the previous state and only print log updates when the detected command actually changes.
- **Bonus 3 — Multiple Objects:** Handle multiple detected target contours in the same frame simultaneously.
- **Bonus 4 — Safety Priority:** If both a Red Square and a Green Circle are detected simultaneously, give obstacle avoidance higher processing priority over safe path indication.

9 Submission Instructions

Submit your work using **GitHub Desktop**:

9.1 1. Create Branch in GitHub Desktop

1. Open GitHub Desktop and select your assignment repository. Use the same personal training repository you set up previously. Do not create a new one.
2. Click **Current Branch** at the top bar and select **New Branch**.
3. Name your branch:

`task-06-vision-avoidance`

4. Create the branch based on `main` and click **Publish branch**.

9.2 2. Implement Your Code

Add all required files locally under the assignment directory. Create a folder named `task-06/` and ensure:

- Your Python script (e.g., `task-06/obstacle_avoidance.py`) runs cleanly with OpenCV installed.

9.3 3. Commit and Push

1. Open GitHub Desktop; you will see all new/modified files listed on the left panel.
2. Ensure all created files are checked.
3. Type a descriptive commit message in the summary box (e.g., `Implement Task 06 vision avoidance script and reflection`).
4. Click **Commit to task-06-vision-avoidance**.
5. Click **Push origin** at the top bar to upload your changes to GitHub.

9.4 4. Open Pull Request

1. In GitHub Desktop, click the **Create Pull Request** button (or press `Ctrl + R` / `Cmd + R`).
 2. This will open your web browser on GitHub.
 3. Ensure base branch is set to `main` and compare branch is `task-06-vision-avoidance`.
 4. Title your Pull Request: `Task 06 -- Vision-Based Obstacle Avoidance`.
 5. Click **Create Pull Request**.
- **Deadline:** September 2, 2026, at 11:59 PM.

Do not merge your Pull Request. The evaluation team will review your submission directly on GitHub.

Build clean code. Test thoroughly. Submit on time.